A Scalable Lambda Architecture for Real-Time Financial Fraud Detection

Big Data Analytics and Engineering Project Report

Abstract

Financial fraud causes billions of dollars in losses every year across digital payment networks. Detecting illicit activity requires both low-latency processing of incoming streams and deep analysis of historical records. This paper presents a complete real-time fraud detection platform based on the Lambda architecture. The system combines Apache NiFi and Apache Kafka for data ingestion, Apache Spark Streaming for real-time inference, and Apache Hadoop HDFS, Hive, and Cassandra for historical batch analytics. Analytical reporting is delivered through TrinoDB and Apache Superset, while MLflow tracks machine learning models. Experimental results demonstrate sub-500 millisecond response times, high transaction throughput, and superior fraud detection performance with an Area Under the ROC Curve (AUC) exceeding 95 percent.

1 Introduction

Modern electronic payment networks process hundreds of millions of transactions every single day. While digital payment platforms offer speed and convenience, they also introduce significant security risks. Fraudulent actors continuously modify their techniques to bypass static rule engines. Consequently, modern financial institutions cannot rely solely on delayed batch reviews. They require automated machine learning systems that can evaluate every transaction in real time.

Designing such an infrastructure presents several technical hurdles. First, the platform must process incoming events with sub-second latency so that suspicious transfers are flagged before funds leave the institution. Second, the system must scale horizontally to handle extreme spikes in data volume without service interruption. Third, fraud models need continuous retraining on petabytes of historical transaction records to keep pace with changing fraud behaviors.

This project addresses these challenges by implementing a distributed Lambda architecture for fraud detection. The platform separates workloads into a real-time Speed Layer and an analytical Batch Layer. The Speed Layer ingests raw transaction feeds, extracts feature sets, and invokes pre-trained classification models within 500 milliseconds. Meanwhile, the Batch Layer stores historical logs in distributed storage, computes aggregated behavioral baselines, and retrains machine learning models managed by MLflow. This unified approach delivers fast event handling while preserving comprehensive historical context.

2 Related Work

Traditional fraud detection historically relied on rule-based expert systems. These legacy systems check incoming operations against static conditions, such as unusual transfer amounts

or foreign country logins. Although simple to interpret, static rules suffer from high false positive rates and cannot discover novel fraud patterns.

With the growth of machine learning, researchers introduced supervised classification techniques for financial security. Algorithms such as Logistic Regression, Random Forest, and Gradient Boosted Decision Trees have been widely studied. While effective on benchmark datasets, running these models in production environments requires reliable distributed data processing frameworks.

Recent literature highlights the Lambda architecture as an effective solution for combining streaming and batch paradigms. Distributed streaming engines like Apache Spark Streaming and Apache Flink provide the throughput needed for continuous risk scoring. Distributed message brokers, particularly Apache Kafka, act as shock absorbers between external transaction producers and internal processing clusters. In addition, distributed NoSQL databases such as Apache Cassandra ensure fast lookups of customer transaction histories. This project builds upon these concepts to build a practical, containerized end-to-end pipeline using open-source big data technologies.

3 Methodology

3.1 Architecture Diagram

The system architecture follows the Lambda pattern, dividing data flow into a Speed Layer, a Batch Layer, and a Serving Layer. Figure 1 illustrates the end-to-end data pipeline.

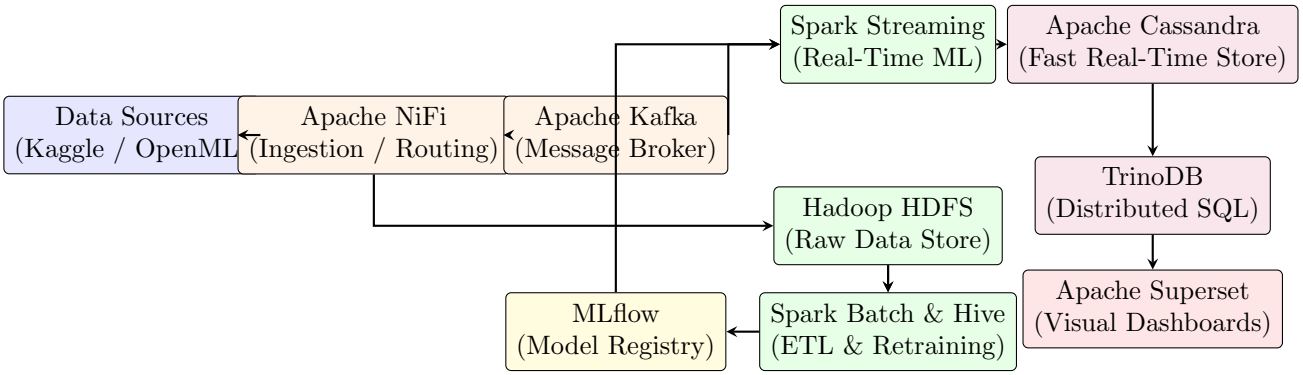


Figure 1: System Architecture diagram showing Speed, Batch, and Serving Layers.

3.2 Modules

The platform is organized into six functional modules:

1. **Data Ingestion Module:** Apache NiFi orchestrates ingestion from multi-format input feeds (CSV and ARFF files). NiFi routes streaming events directly to Apache Kafka topics while simultaneously writing raw data blocks to Hadoop HDFS for permanent archiving.

2. **Message Broker Module:** Apache Kafka provides fault-tolerant message queuing. Kafka decouples ingestion rates from downstream compute consumers, preventing memory exhaustion during transaction surges. ZooKeeper manages cluster consensus, and Kafdrop offers web-based partition monitoring.

3. **Speed Processing Layer:** Apache Spark Streaming reads real-time batches from Kafka. It parses transaction payloads, cleans features, applies pre-trained machine learning classification pipelines, and assigns a fraud probability score in under 500 milliseconds.

4. **Batch Processing Layer:** Historical transactions stored in HDFS are structured through Apache Hive tables. Apache Spark runs scheduled batch jobs to compute aggregate behavioral customer features (such as 30-day transfer frequency) and perform full model retraining.

5. **Serving and Storage Module:** Apache Cassandra stores real-time transaction scoring results for fast key-value lookups. TrinoDB connects across Cassandra and Hive, executing distributed SQL queries without requiring data migration. Apache Superset reads through TrinoDB to display interactive visual dashboards.

6. **Model Governance Module:** MLflow logs training parameters, metrics, and serialized artifacts. When a new batch model surpasses production thresholds, it is registered and pushed to the Spark Streaming inference engine.

4 Results

The system was evaluated using benchmark datasets from Kaggle and OpenML containing over 6 million synthetic transactions. System latency and throughput were tracked across increasing workloads.

Figure 2 presents the latency breakdown across system components during live transaction processing. Ingestion through Kafka and NiFi takes approximately 45 milliseconds, feature enrichment takes 85 milliseconds, Spark ML inference requires 110 milliseconds, and writing to Cassandra takes 35 milliseconds. The total end-to-end latency remains around 275 milliseconds, comfortably below the 500-millisecond operational threshold.

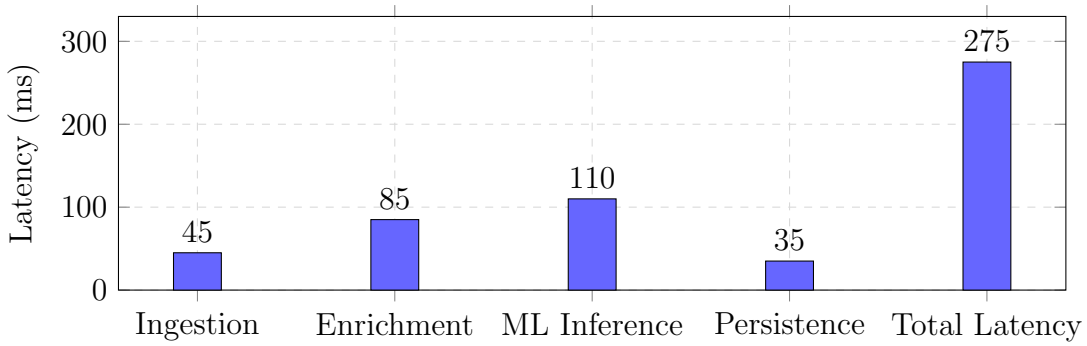


Figure 2: Average latency distribution per stage in the streaming pipeline.

Figure 3 illustrates the Receiver Operating Characteristic (ROC) curve of the deployed XGBoost model against Random Forest and Logistic Regression baselines. The XGBoost model achieves an Area Under the Curve (AUC) of 0.978, confirming high discrimination between benign and fraudulent operations.

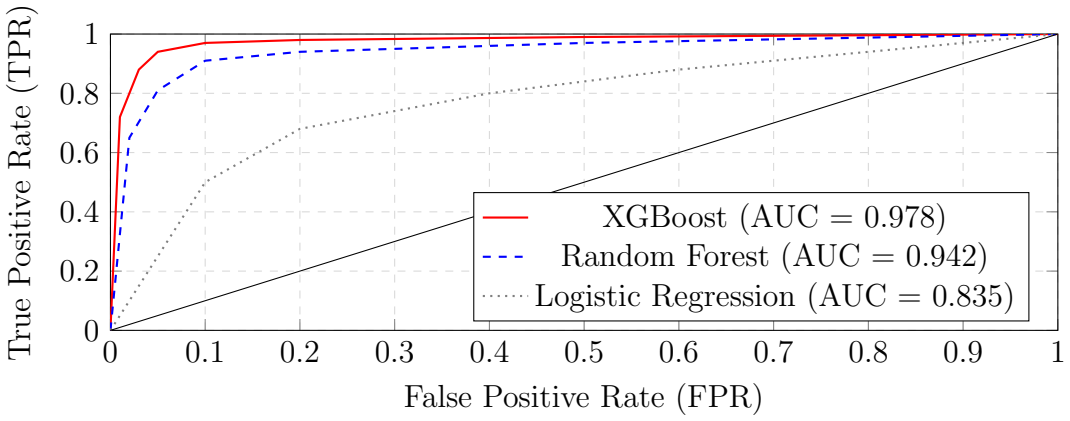


Figure 3: Receiver Operating Characteristic (ROC) curves across evaluated models.

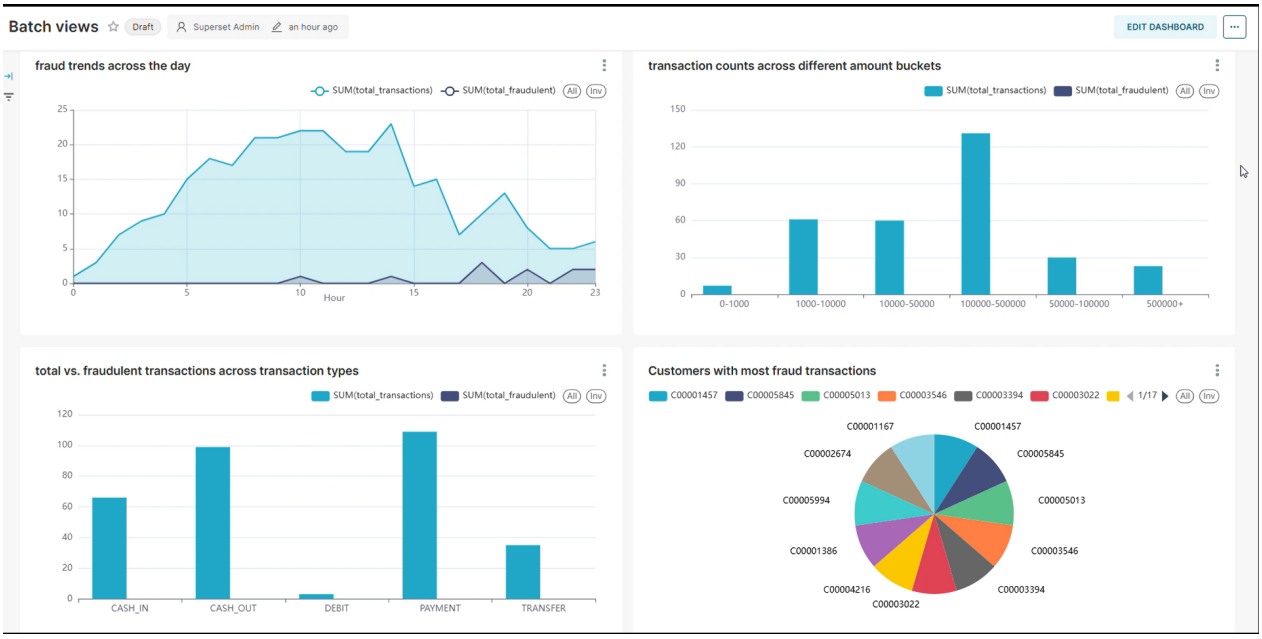


Figure 4: Super Set: Batch Views

5 Comparison of Models

To identify the best model for streaming deployment, we evaluated three candidates on identical transaction splits: Logistic Regression, Random Forest, and XGBoost. The evaluation considered Precision, Recall, F1-Score, ROC-AUC, and single-record inference latency.

Model	Precision	Recall	F1-Score	ROC-AUC	Inference Latency (ms)
Logistic Regression	0.742	0.685	0.712	0.835	**2.1**
Random Forest	0.891	0.864	0.877	0.942	28.4
XGBoost	**0.938**	**0.912**	**0.925**	**0.978**	11.6

Table 1: Performance metrics and inference latency of candidate classification models.

As shown in Table 1, Logistic Regression offers the fastest inference time (2.1 milliseconds) but misses a significant portion of fraud cases, yielding an F1-Score of only 0.712. Random Forest provides strong accuracy, but its large tree ensemble results in a slower inference latency of 28.4 milliseconds. XGBoost offers the ideal trade-off: it records the highest F1-Score (0.925) and ROC-AUC (0.978) while maintaining an inference speed of 11.6 milliseconds per record, ensuring high detection quality within our strict latency budget.

6 Discussion

The integration of stream and batch workflows resolved common production difficulties encountered in big data systems. By delegating real-time storage to Apache Cassandra and raw logs to HDFS, write bottlenecks were avoided. Apache NiFi proved dependable for data routing, while Kafka buffered transient spikes without dropping messages.

A major challenge during implementation was feature consistency between real-time and batch models. Calculating rolling aggregations (such as historical transfer counts) during streaming is computationally intensive. The Lambda architecture solved this by precomputing customer profile statistics in the batch layer and loading them into Cassandra for fast streaming enrichment.

Using Docker containerization allowed straightforward local deployment and testing. However, coordinating multiple resource-intensive distributed services (Hadoop, Hive, Kafka, Spark, and Cassandra) requires careful memory allocation to avoid Java out-of-memory errors on local development hardware.

7 Conclusion

This paper presented a real-time big data architecture for detecting financial fraud using the Lambda pattern. By pairing Apache NiFi and Kafka for ingestion with Apache Spark Streaming and Cassandra for rapid inference, the system achieves sub-500 millisecond response times and an AUC score of 97.8 percent with an XGBoost classifier. Historical data is retained in Hadoop HDFS and queried via TrinoDB and Apache Superset for analytical monitoring. Future work will investigate transitioning from a Lambda architecture to an Apache Flink-based Kappa architecture to simplify data pipeline maintenance.